

Padronização e Qualidade de Código Profissional

1. Organização física não garante qualidade interna

A organização por pacotes representa maturidade estrutural.

Entretanto, estrutura externa não garante qualidade interna.

Um projeto pode possuir:

- pacotes organizados,
- separação de responsabilidades,
- importações hierárquicas corretas,

e ainda assim conter código mal escrito internamente.

A organização física resolve o “onde”. A padronização resolve o “como”.

Essa distinção é fundamental.

Arquitetura externa define camadas.

Qualidade interna define legibilidade, previsibilidade e manutenção.

2. O problema do código mal escrito

Um código mal escrito não significa que ele está “quebrado”. Significa que ele dificulta leitura, manutenção e evolução.

Considere o exemplo abaixo:

```
def f(a, b):  
    if a != "":  
        if b != "":  
            if a.isalpha():  
                if b.isdigit():  
                    return True  
        return False  
    return False
```

Funciona?

Sim.

É claro?

Não.

Problemas identificáveis:

- nome da função não comunica intenção;
- variáveis genéricas (a, b);
- validações encadeadas desnecessariamente;
- leitura vertical difícil;
- ausência de responsabilidade explícita.

Código que funciona não é sinônimo de código profissional.

3. Código reorganizado com qualidade estrutural

Agora o mesmo comportamento, porém com padronização adequada:

```
def validar_nome_e_idade(nome: str, idade_texto: str) -> bool:
    if not nome.strip():
        return False

    if not idade_texto.strip():
        return False

    if not nome.isalpha():
        return False

    if not idade_texto.isdigit():
        return False

    return True
```

Diferenças observáveis:

- nome da função comunica intenção;
- parâmetros possuem nomes significativos;
- uso de tipagem explícita;

- validações separadas por responsabilidade;
- fluxo de leitura linear e claro.

O comportamento é o mesmo.

A qualidade é completamente diferente.

4. Nomeação coerente (snake_case)

A padronização básica de Python utiliza:

- letras minúsculas
- separação por underscore (`_`)
- nomes descritivos

Exemplos adequados:

- `validar_nome`
- `calcular_media`
- `processar_pagamento`
- `exibir_resultado`

Exemplos problemáticos:

- `Funcao1`
- `validarNome`
- `v`
- `processarDadosDoSistemaPrincipalCompletoFinal2`

Nomeação deve equilibrar:

- clareza
- concisão
- coerência

Nome é documentação embutida no código.

5. Funções com responsabilidade única

Um dos princípios mais importantes da qualidade interna é:

Uma função deve fazer apenas uma coisa.

Exemplo inadequado:

```
def cadastrar_usuario(nome, idade):
```

```
    if not nome:
```

```
        print("Nome inválido")
```

```
        return
```

```
    if not idade.isdigit():
```

```
        print("Idade inválida")
```

```
        return
```

```
    print("Usuário cadastrado")
```

Problema:

- validação
- processamento
- exibição

Tudo misturado.

Agora versão organizada:

```
def validar_nome(nome: str) -> bool:
```

```
    return bool(nome.strip())
```

```
def validar_idade(idade: str) -> bool:
```

```
    return idade.isdigit()
```

```
def processar_cadastro(nome: str, idade: str) -> dict:
```

```
    if not validar_nome(nome):
```

```
        return {"sucesso": False, "mensagem": "Nome inválido."}
```

```
    if not validar_idade(idade):
```

```
        return {"sucesso": False, "mensagem": "Idade inválida."}
```

```
return {"sucesso": True, "mensagem": "Cadastro realizado."}
```

Separação clara de responsabilidades. Isso facilita:

- testes
- manutenção
- reutilização

6. Evitar código solto

Código solto é aquele que executa fora de função sem necessidade.

Exemplo inadequado:

```
print("Iniciando sistema")
nome = input("Nome: ")
print("Finalizando")
```

Versão organizada:

```
def main():
    print("Iniciando sistema")
    nome = input("Nome: ")
    print("Finalizando")

if __name__ == "__main__":
    main()
```

Organização:

- fluxo centralizado
- execução controlada
- possibilidade de reutilização

7. Organização interna do arquivo

Uma estrutura profissional mínima dentro de um arquivo Python deve seguir ordem lógica:

1. Importações
2. Constantes
3. Funções auxiliares
4. Função principal
5. Bloco de execução (if `__name__ == "__main__":`)

Exemplo estruturado:

```
# 1. Imports
import datetime

# 2. Constantes
FORMATO_DATA = "%d/%m/%Y"

# 3. Funções auxiliares
def obter_data_atual():
    return datetime.datetime.now().strftime(FORMATO_DATA)

# 4. Função principal
def main():
    data = obter_data_atual()
    print("Data atual:", data)

# 5. Execução
if __name__ == "__main__":
    main()
```

Essa organização cria previsibilidade. Previsibilidade reduz erro.

8. Evitar variáveis globais desnecessárias

Variáveis globais aumentam acoplamento.

Exemplo inadequado:

```
contador = 0
```

```
def incrementar():  
    global contador  
    contador += 1
```

Problema:

- dependência externa invisível;
- difícil rastrear modificações.

Versão melhor:

```
def incrementar(contador: int) -> int:  
    return contador + 1
```

Função pura, previsível.

9. Importações organizadas no topo

Importações devem estar:

- no início do arquivo;
- agrupadas;
- separadas de lógica.

Evitar:

```
def funcao():  
    import math  
    print(math.sqrt(4))
```

Preferir:

```
import math
```

```
def funcao():  
    print(math.sqrt(4))
```

Organização interna também é organização mental.

10. Mentalidade profissional

Código é lido mais vezes do que escrito.

Manutenção é parte inevitável do ciclo de vida de um sistema.

Em ambientes profissionais:

- múltiplos desenvolvedores trabalham no mesmo projeto;
- mudanças são constantes;
- erros custam tempo e dinheiro.

Padronização não é estética.

Padronização é estratégia de sustentabilidade técnica.

11. Comparação final — Código desorganizado vs organizado

Versão desorganizada

```
def calc(a,b):  
    return a+b
```

```
x=10  
y=20  
print(calc(x,y))
```

Versão organizada


```
def calcular_soma(valor_a: int, valor_b: int) -> int:
    return valor_a + valor_b

def main():
    valor_a = 10
    valor_b = 20
    resultado = calcular_soma(valor_a, valor_b)
    print(resultado)

if __name__ == "__main__":
    main()
```

Diferença estrutural:

- nome significativo
- tipagem explícita
- separação de fluxo
- clareza de intenção

12. Justificativa estrutural

Estrutura externa (pacotes) organiza o projeto.

Qualidade interna organiza o código dentro do projeto.

Ambos são necessários. Sem qualidade interna:

- arquitetura externa perde força.

Sem arquitetura externa:

- qualidade interna fica isolada e pouco escalável.

A combinação de:

- pacotes hierárquicos
- nomeação coerente
- responsabilidade única
- organização interna previsível

forma a base mínima de maturidade profissional em Python.